Programming Assignment Four

Brynn Grofcsik, Bijan Moradi

Department of Computer Science & Information Systems, CSU San Marcos

CS 433: Operating Systems

Dr. Xiaoyu Zhang

November 14, 2025

Problem Description

Our program aims to solve the consumer-producer problem in the context of a bounded buffer. This is a common synchronization problem described often in the field of computer science. The consumer-producer problem consists of multiple processes or threads that communicate using shared memory. It is critical that reading and writing to the shared memory be tightly regulated and synchronized in order to prevent memory corruption and resulting segmentation faults. Our program will implement an integer buffer representing memory items, as well as consumer and producer processes using the POSIX pthread API. We will use Semaphores to synchronize the processes.

Program Design

Buffer: Our buffer consisted of a dynamically allocated array of buffer_item (Integer type) pointers. We implemented basic element addition, removal, as well as a constructor and destructor. We chose to dynamically allocate the array because numerous items will be entering and leaving the buffer.

Producer: Our producer utilizes the Semaphore.h library to implement synchronization between our consumer and producer threads. The producer interacts with an instantiated Buffer class, which contains the shared memory of the process.

Consumer: Similarly, our consumer utilizes the Semaphore.h library to implement synchronization between our consumer and producer threads. The consumer interacts with an instantiated Buffer class, which contains the shared memory of our process.

System Implementation

- Buffer.h
    - Contains header declarations of functions used within the program.
- Buffer.cpp
    - get_size()
        - Returns the buffer size.
    - get_count()
        - Returns the buffer count.
    - is_empty()
        - Checks if the buffer count is equal or less than zero. If it is empty, it returns a Boolean value of true.

    - is_full()
        - Checks if the buffer is full by comparing the buffer count to the buffer's limited size. If it is full, it returns a Boolean value of true.
    - insert_item()
        - Adds a new item to the buffer. It first checks if the buffer is full; if it isn't, it adds a given buffer item to the back of the buffer.
    - remove_item()
        - Removes the first item from our Buffer and shifts the remaining items by one to the left within the buffer.
    - print_buffer()
        - Prints the current list of items within our buffer by creating a string that appends each buffer_item (after converting it to a substring and adding a ",").
    - getBufferFront()
        - Returns the first-in item from our Buffer, a vital function considering our algorithm is FIFO-based
- Main.cpp
    - Producer()
        - Creates a new buffer item through a producer thread's insertion of its own ID within the function. It temporarily sleeps before proceeding, locks both the empty semaphore and the mutex lock, and then adds an item. After exiting this critical section, it frees up both the empty semaphore and mutex lock.
    - Consumer()
        - Grabs the first entered buffer item through getBufferFront() and consumes it. It temporarily sleeps before proceeding, locks both the full semaphore and mutex lock, and then consumes the first entered item. After exiting this critical section, it frees up both the full semaphore and the mutex lock.
    - Main()
        - Processes given command lines by dissecting them into a sleep int, producer thread count int and consumer thread count int. Afterwards, it initializes the mutex lock, empty semaphore and full semaphore. The function then creates a set of producer and consumer threads dependent on the int value entered by the user previously. The program then sleeps for a period of time and then exits.

Issues:

We didn't run into many issues during the development of our program. The majority of our issues were small oversights quickly caught by the other programmer upon examination (such as mistakenly writing exit(1) rather than exit(0). We did run into some issues trying to

synchronize our program within Gradescope, but after a few internal adjustments (such as fixing the previously mentioned exit error) and a correct implementation of FIFO using a newly specialized function (getBufferFront()), we were able to get it operating as intended.

Results

An example of our program running as intended using the following input:

./prog4 4 3 2

4 = the number of seconds the main thread sleeps before terminating

3 = the number of producer threads created

2 = the number of consumer threads created

```
brynn@DESKTOP-D5RI9HB:/mnt/c/Users/eonfl/Documents/GitHub/cs433_assign_starter_FALL2025/assign4$
 ./prog4 4 3 2
Producer 1: Inserted item 1
Buffer: [1]
Producer 1: Inserted item 1
Buffer: [1, 1]
Consumer Removed item 1
Buffer: [1]
Producer 3: Inserted item 3
Buffer: [1, 3]
Consumer Removed item 1
Buffer: [3]
Producer 2: Inserted item 2
Buffer: [3, 2]
Producer 2: Inserted item 2
Buffer: [3, 2, 2]
Producer 3: Inserted item 3
Buffer: [3, 2, 2, 3]
Consumer Removed item 3
Buffer: [2, 2, 3]
Consumer Removed item 2
Buffer: [2, 3]
Producer 1: Inserted item 1
Buffer: [2, 3, 1]
Producer 3: Inserted item 3
Buffer: [2, 3, 1, 3]
Producer 2: Inserted item 2
Buffer: [2, 3, 1, 3, 2]
Consumer Removed item 2
Buffer: [3, 1, 3, 2]
Producer 3: Inserted item 3
Buffer: [3, 1, 3, 2, 3]
```

```
Consumer Removed item 3
Buffer: [1, 3, 2, 3]
Producer 1: Inserted item 1
Buffer: [1, 3, 2, 3, 1]
Consumer Removed item 1
Buffer: [3, 2, 3, 1]
Producer 2: Inserted item 2
Buffer: [3, 2, 3, 1, 2]
Consumer Removed item 3
Buffer: [2, 3, 1, 2]
Producer 3: Inserted item 3
Buffer: [2, 3, 1, 2, 3]
Consumer Removed item 2
Buffer: [3, 1, 2, 3]
Producer 3: Inserted item 3
Buffer: [3, 1, 2, 3, 3]
Consumer Removed item 3
Buffer: [1, 2, 3, 3]
Producer 1: Inserted item 1
Buffer: [1, 2, 3, 3, 1]
Consumer Removed item 1
Buffer: [2, 3, 3, 1]
Producer 2: Inserted item 2
Buffer: [2, 3, 3, 1, 2]
Consumer Removed item 2
Buffer: [3, 3, 1, 2]
Producer 3: Inserted item 3
Buffer: [3, 3, 1, 2, 3]
Consumer Removed item 3
Buffer: [3, 1, 2, 3]
Producer 1: Inserted item 1
Buffer: [3, 1, 2, 3, 1]
```

```
Consumer Removed item 3
Buffer: [1, 2, 3, 1]
Producer 2: Inserted item 2
Buffer: [1, 2, 3, 1, 2]
```

Conclusion

We were able to successfully implement a program that solved the producer/consumer problem. We were able to successfully simulate a system of producer and consumer threads that accessed a buffer without any issues arising due to race conditions through the usage of counting semaphores and a mutex lock. If we were to revisit this program in the future, we would be

interested in adding additional variables to the buffer item objects to see how that would affect the execution.